Collaborative Large and Small Language Models for Accurate and Scalable Data Repair

Qian Chen, Jianwei Wang, Wenjie Zhang
University of New South Wales
Sydney, Australia
{qian.chen6,jianwei.wang1,wenjie.zhang}@unsw.edu.au

***Abstract*—We study the problem of data repair, a key task in data cleaning that corrects erroneous entries in raw datasets to improve overall data quality. Although recent data-driven methods, especially those based on large language models (LLMs), achieve remarkable performance, we observe that: (i) they directly repair data in the raw and low-quality context, which may compromise learning signals, and (ii) they directly use uncertain model outputs as repairs, potentially introducing unreliable corrections and compromising repair quality. Motivated by the efficiency of small language models (SLMs) and the capabilities of LLMs, and aiming to address the above limitations, we propose LasRepair, a framework that collaborates Large and small language models for data Repair. LasRepair employs an LLM as an instructor, which selects a global repair context to guide the SLM. The SLM acts as a corrector, using the selected context to repair erroneous data more efficiently. Moreover, to further improve context quality, we extend LasRepair to LasRepair+, which formulates data repair as an Expectation-Maximisation (EM) procedure that alternates between an E-step for updating the corrector parameters and an M-step for refining the repair context. Furthermore, to mitigate model uncertainty, we propose LasRepair++, which uses column-calibrated model confidence to down-weight unreliable repaired rows when updating the corrector, thereby enhancing repair quality. Theoretical analysis and empirical evaluation demonstrate the superiority of our methods. We theoretically prove the effectiveness of the EM-style procedure and the confidence-based weighting. Experiments on real-world datasets show that LasRepair++ achieves an average F1-score improvement of 18.1% over the strongest baseline. Code is available at https://github.com/T-Lab/LasRepair.**

***Index Terms*—Data repair, data cleaning, large language models, expectation-maximisation, confident learning**

I. Introduction

Real-world datasets are often noisy and error-prone, containing errors such as missing values, typos, and other inconsistencies, as illustrated in Figure 1. Such low-quality data can negatively affect the training of downstream models [1], especially large language models (LLMs), which are sensitive to data quality [2], [3]. To address this issue, data cleaning has been widely studied as a fundamental task in data quality, aiming to correct erroneous entries in relational tables and improve the reliability of real-world datasets [4], [5]. Data cleaning typically consists of two tasks: error detection and data repair (i.e., error correction). Error detection aims to identify erroneous data entries, and data repair focuses on correcting them once they have been detected. In this work, we focus on data repair and aim to improve data quality.

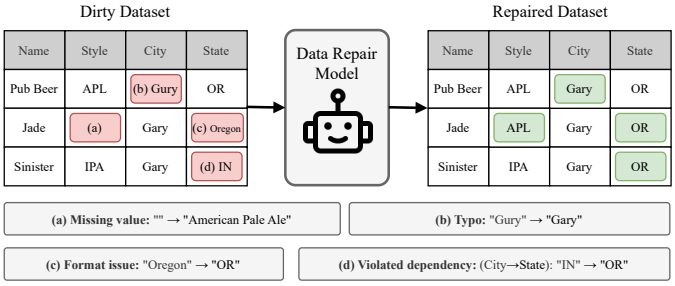

Fig. 1. An illustrated example of common data errors. The dirty table contains (a) a missing value $x_2[Style]$, (b) a typo in $x_2[City]$, (c) a formatting issue in $x_1[State]$, and (d) an attribute-dependency violation (e.g., $City \rightarrow State$) reflected in $x_3[State]$.

The problem of data repair has been studied extensively in the data management field (see the survey paper [6] for a more comprehensive introduction). Conventional data repair methods mainly include *constraint-driven* methods [7]–[11] and *hybrid* methods [12]–[14] that leverage integrity constraints, statistical distributions, or downstream model performance to generate repair candidates and select repairs based on specific optimization criteria [7], [12], [15]. However, these methods typically rely on handcrafted rules or task-specific assumptions, which limit their adaptability to complex, heterogeneous data and restrict their effectiveness when constraints are incomplete, noisy, or difficult to specify in practice.

Recently, *data-driven* methods [16]–[19] have emerged as a more flexible alternative, learning to propose and select corrections directly from data and achieving state-of-the-art (SOTA) performance. These methods can be broadly categorized into discriminative classifier-based methods and generative LLM-based methods. Discriminative methods, exemplified by Baran [17] in Figure 2(a), generate repair candidates from multiple contextual views and train lightweight classifiers to select plausible corrections with limited supervision. Generative LLM-based methods, exemplified by Jellyfish [19] and GIDCL [18] in Figure 2(b), exploit the semantic capabilities and world knowledge of LLMs for generative data repair.

Motivation. Although these data-driven methods achieve SOTA accuracy, two crucial limitations persist.

First, existing repair methods are affected by *context contamination*. These methods are trained on raw datasets that may contain an unavoidable fraction of errors. As a result, the context used to repair one cell can contain misleading

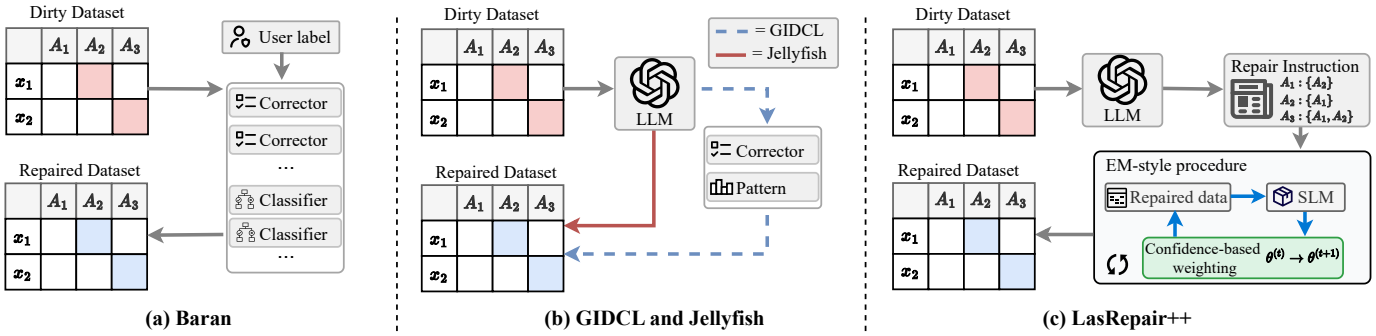

Fig. 2. Conceptual comparison of representative data repair methods. (a) Baran relies on user-provided labels to train correctors and classifiers to produce a repaired dataset. (b) GIDCL and Jellyfish incorporate LLMs either to assist in correction pattern discovery or to perform direct repair. (c) LasRepair++ generates repair instructions with an LLM instructor and performs an EM-style procedure with an SLM corrector, further incorporating confidence-based weighting to improve performance.

information. This issue is particularly severe when the error rate is high or when attributes are strongly correlated. To mitigate this problem, Baran [17] relies on user-labeled examples as clean supervision. However, such labels are costly and typically limited in scale, making it difficult to construct consistently reliable repair contexts.

Second, they may directly use uncertain model outputs as repairs. Such uncertainty can arise across both model training and inference. During training, these repair models often use stochastic algorithms to optimize the loss, such as SGD [20], which optimizes the loss through randomized gradient updates, introducing uncertainty into the learned parameters. For inference, these repair models usually generate probability distributions over possible outputs and commit to high-probability candidates, discarding the remaining probability mass. For example, Jellyfish [19] repairs serialized instances through autoregressive generation, where uncertainty in token-level probability distributions can accumulate across decoding steps and lead to uncertain repaired values in the final output [21].

Challenges. To overcome these limitations and design an effective data repair method, two main challenges exist.

Challenge I: How to find a reliable context in the low-quality table with complex relationships? Real-world tables often contain various types of errors across multiple attributes that contaminate the repair context, making reliable context selection difficult. A straightforward strategy is to remove all erroneous values, but such aggressive filtering can discard informative attributes or tuples, leading to insufficient context and biased repairs [22]. Another promising direction is to select correlated attributes as context to balance error reduction and information preservation [18], [23], [24]. Such a trade-off is difficult to achieve in complex relational tables, where useful dependencies may be implicit and entangled with corrupted values [12]. Therefore, finding reliable context from a low-quality table remains a challenge.

Challenge II: How to effectively and efficiently alleviate model uncertainty on large datasets? On large datasets, alleviating model uncertainty is challenging in terms of both effectiveness and efficiency [25]. On the one hand, effectiveness is hard to guarantee because models trained on large datasets can exhibit high prediction variance, leading to unstable and inconsistent repair outputs [20], [26]. On the other hand, maintaining efficiency at scale is challenging because the number of cells requiring repair grows with dataset size, making even lightweight repair operations computationally costly. Therefore, effectively reducing uncertainty while avoiding additional verification or external checking remains a key challenge for large-scale data repair.

Our approaches. Motivated by the above challenges and the observation that small language models (SLMs) offer efficiency and LLMs provide strong reasoning capabilities, we propose LasRepair, a **L**arge **a**nd **s**mall language model for accurate and scalable data **Repair**. LasRepair adopts the *LLM-as-an-instructor* paradigm [27], in which an LLM instructs a corrector (i.e., an SLM) to enable scalable data repair. Specifically, instead of relying on an LLM to repair every erroneous cell, LasRepair leverages the strong semantic reasoning capabilities and broad prior knowledge of the LLM [28] to derive compact and relevant repair contexts for each target column. Based on these contexts, the corrector is fine-tuned as a cell-level sequence-to-sequence repair model.

Based on LasRepair and to further address *Challenge I*, we propose LasRepair+. LasRepair+ formulates data repair as an Expectation–Maximization (EM) process [29]–[31] and iteratively performs the maximization step (M-step) and expectation step (E-step) to progressively improve the context quality and repair accuracy [32]. In the M-step, the corrector infers candidate repairs for the dataset and writes the generated repair back to the dataset to improve context quality. In the E-step, the updated context is used as input to further fine-tune the corrector, enabling it to learn from increasingly reliable repaired data. The updated corrector subsequently produces updated repairs for the next M-step.

Building on LasRepair+, we introduce LasRepair++ to address *Challenge II* by assigning lower weights to potentially low-quality repairs during the E-step. Rather than treating all generated repairs equally, LasRepair++ estimates the reliability of generated repairs and incorporates this reliability into model updates [33]. Specifically, it derives confidence-based weights from model predictions and uses them to mitigate model uncertainty. This lightweight weighting process reuses existing model outputs without requiring additional model calls, thereby avoiding substantial computational overhead.

Consequently, repairs with lower weights (higher uncertainty) contribute less to corrector updates.

Theoretical and empirical studies. We theoretically and empirically demonstrate the superiority of our methods. Theoretically, we analyze the proposed refinement procedure and show the monotonic behavior of the corresponding objective. Moreover, we analyze how confidence-based weighting can reduce the influence of noise under a heteroscedastic noise model. Empirical evaluations on 7 real-world datasets also demonstrate the strong performance of LasRepair++ in terms of 3 metrics. LasRepair++ achieves F1-score improvement ranging from 5.6% to 26.8% over the previous SOTA methods. LasRepair++ also achieves the highest error drop rate on 6 out of 7 datasets and yields an average gain of 16% over the best baseline. Furthermore, LasRepair++ can efficiently handle large-scale datasets without compromising accuracy.

Contributions. The key contributions of this paper are summarized as follows.

- We present the LasRepair family for data repair. The base LasRepair separates global context selection from local value generation, using an LLM as a dataset-level *instructor* and a fine-tuned SLM as a cell-level *corrector*.
- We propose an EM-style procedure that jointly updates the repaired data and the SLM corrector, allowing contaminated contexts to be progressively improved.
- We develop a confidence-based weighting mechanism that estimates the reliability of generated repairs and incorporates row-level confidence weights into SLM training, reducing the influence of model uncertainty.
- We demonstrate the effectiveness of LasRepair++ empirically through extensive experiments on 7 benchmark datasets, where it consistently outperforms SOTA baselines, yielding significant F1-score improvements and strong robustness across varying noise levels.

II. PRELIMINARIES

In this section, we formally define the problem and then review the SOTA solutions and their limitations. Frequently used notations are summarized in Table I.

A. Problem definition

Following prior work [6], [17], [18], we consider a relational table with n records and d attributes. Let $\mathcal{X}_j$ denote the domain of attribute j, and let $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$ denote the tuple (record) domain. The observed dirty table is $\mathbf{D}_e \in \mathcal{X}^n$ and the (unknown) clean ground-truth table is $\mathbf{D}_c \in \mathcal{X}^n$. Let $\mathbf{x}_i = (x_{i1}, \ldots, x_{id})$ be the i-th record in $\mathbf{D}_e$, where x_{ij} denotes the value of cell (i, j) and z_{ij} denotes its ground-truth value. For readability, we use column names $\mathcal{A} = \{a_1, \ldots, a_d\}$ and write $x_i[a_j] \triangleq x_{ij}$ (similarly $z_i[a_j] \triangleq z_{ij}$). For iterative repair, let $\mathbf{D}_r^{(t)}$ be the repaired table at iteration t with $\mathbf{D}_r^{(0)} = \mathbf{D}_e$, and let $\mathbf{D}_r^{(T)}$ denote the final repaired table.

A cell (i, j) is *erroneous* if $x_{ij} \neq z_{ij}$. The unknown true error set is denoted by $\mathcal{E}^* = \{(i, j) \mid x_{ij} \neq z_{ij}\}$.

In practice, $\mathbf{D}_c$ is unavailable, and an error detector returns a detected error set $\mathcal{E} \subseteq [n] \times [d]$, which specifies the cells to be repaired. This work focuses on the repair stage and assumes that $\mathcal{E}$ is given by an oracle detector.

TABLE I
Summary of the main symbols and definitions used in this paper.

Notation	Description
$\mathbf{D}_e, \mathbf{D}_c, \mathbf{D}_r^{(t)}$	Dirty/clean/repaired dataset.
x_i, x_{ij}	Tuple i and cell (i, j).
$z_{ij}, \hat{z}_{ij}^{(t)}$	Clean/repaired value at iteration t.
$\mathcal{E}$	Set of detected erroneous cells.
C	Column set.
S	Dataset sketch for the instructor LLM.
$\mathbf{W}$	Column influence matrix.
$\mathcal{G}$	Induced column graph.
$\mathcal{T}_j$	Influence tree for target column j.
$N(j)$	Selected columns for target column j.
$\mathbf{x}_i^{(j)}$	Serialized input for repairing cell (i, j).
g_θ	SLM repair model.
$\theta^{(t)}$	Model parameters at iteration t.
$\omega_i^{(t)}$	Row-level weight at iteration t.
$\omega_i^{*(t)}$	Normalized tuple weight at iteration t.

Definition 1 (Data repair)**.** Given a dirty table $\mathbf{D}_e$ and a detected error set $\mathcal{E}$, the goal of data repair is to produce a repaired table $\mathbf{D}_r$ that modifies only the detected erroneous cells and restores the erroneous values as closely as possible to their unknown clean values:

$$\mathbf{D}_r[i, j] = x_{ij} \; \forall (i, j) \notin \mathcal{E}, \quad \min_{\mathbf{D}_r} \sum_{i=1}^{n} \sum_{j=1}^{d} \mathbb{I}[\mathbf{D}_r[i, j] \neq \mathbf{D}_c[i, j]] \, .$$

B. State-of-the-art

As indicated by prior work and recent surveys [6], SOTA data-repair performance is increasingly achieved by data-driven methods that learn repair decisions from data rather than relying solely on manually specified constraints. Representative methods can be broadly grouped into two categories: (i) *discriminative, classifier-based* methods that select repairs from a candidate set (e.g., Baran), and (ii) *generative, LLM-based* methods that directly generate repairs using the capabilities of LLMs (e.g., GIDCL and Jellyfish). For simplicity, we refer to them as discriminative and generative methods.

Discriminative methods. Baran [17] is a representative discriminative repair framework. Given a dirty table and a set of detected erroneous cells, Baran instantiates multiple correctors that exploit complementary contextual signals of a cell to propose candidate repairs. It then represents each cell-candidate pair with aggregated features and trains lightweight classifiers from limited user-provided corrections to select the most plausible candidate.

Generative methods. Recent generative methods formulate repair as a sequence generation problem. GIDCL [18] combines graph-based table modeling with LLM-based repair, using structural signals to assist correction pattern discovery and improve repair consistency. Jellyfish [19] formulates data repair as an instruction-following generation task, where a serialized table context and error information are provided to a generative model to produce corrected values.

III. OUR METHOD

In this section, we introduce the LasRepair family, which consists of three progressively enhanced variants. LasRepair is the base instructor–corrector framework in which an LLM instructor provides table-level structural guidance and an SLM corrector performs scalable cell-level repair. LasRepair+ extends LasRepair with an EM-style procedure that repeatedly updates the repaired table and the corrector. LasRepair++ further extends LasRepair+ with confidence-based weighting to reduce the influence of model uncertainty.

A. LasRepair method

Motivation. A single model that directly repairs all cells often faces a trade-off between global reasoning and scalability. This trade-off can be addressed by leveraging language models. LLMs provide strong semantic reasoning, but applying an LLM to every erroneous cell can be costly, especially on large tables. SLMs are more efficient and easier to fine-tune, but they require compact and informative context. LasRepair therefore adopts an instructor–corrector paradigm: the instructor provides global, table-level structural guidance, while the corrector performs efficient cell-level repair.

LasRepair consists of two tightly connected components: (i) an instructor model that infers a column influence structure to select a context set $\mathcal{N}(j)$ for each target column j; (ii) a corrector model g_θ that repairs erroneous cells in $\mathcal{E}$ conditioned on the selected context. This design separates global context selection from local value generation, reducing irrelevant or weakly informative context while preserving scalability.

LLM-as-an-instructor. Structural decisions in LasRepair are made by an instructor at the table level. Let $\mathcal{C} = \{1, \ldots, d\}$ denote the set of column indices. We construct a compact table sketch $\mathbf{S}$ consisting of column names and a small set of rows sampled from $\mathbf{D}_e$, and then query the instructor once to infer inter-column informativeness. The instructor returns a column influence matrix $\mathbf{W} \in [0,1]^{d\times d}$ with entries

$$\mathbf{W}_{ab} \;=\; f_{\text{LLM}}(a, b, \mathbf{S}) \in [0,1], \qquad a \neq b,$$

where $\mathbf{W}_{ab}$ quantifies the informativeness of column a when repairing column b. We interpret $\mathbf{W}$ as a directed weighted column influence graph:

$$\mathcal{G} = (\mathcal{V}, \mathcal{A}, \mathbf{W}), \quad \mathcal{V} = \mathcal{C}, \quad \mathcal{A} = \{(a \to b) : a, b \in \mathcal{V}, a \neq b\}.$$

Although $\mathbf{W}$ is defined pairwise, repairing a target column may require context from multiple correlated columns. We therefore aggregate influence along directed paths that lead to the target column. For each target column $j \in \mathcal{C}$, and each candidate context column $v \neq j$, we define

$$\phi_j(v) = \max_{P:\, v \rightsquigarrow j,\, |P| \le h} \prod_{(u\to u') \in P} \mathbf{W}_{uu'},$$

where P ranges over directed paths from v to j with length at most h. Intuitively, $\phi_j(v)$ measures the strongest path through which column v can inform the repair of column j, either directly or indirectly. Given a threshold $\epsilon > 0$, we then select the context set as

$$\mathcal{N}(j) = \{\, v \in \mathcal{V} \setminus \{j\} :\; \phi_j(v) \ge \epsilon \,\}.$$

By default, we set $\epsilon = 0.6$ to control the context size. For interpretability, each selected $v \in \mathcal{N}(j)$ is connected to j through the strongest path $\phi_j(v)$, resulting in an h-hop influence tree rooted at j. This produces a deterministic and consistent context set for the corrector to use.

SLM-as-a-corrector. Given the context set $\mathcal{N}(j)$ for each target column j, we formulate cell repair as a sequence generation problem. For every row i and target j, we construct a textual input sequence $\mathbf{x}_i^{(j)}$ as follows:

$$\mathbf{x}_i^{(j)} = \text{Serialize}\Big(\{(a_k, x_{ik}) : k \in \mathcal{N}(j)\},\, a_j\Big),$$

where $\text{Serialize}(\cdot)$ maps the selected column-value pair (a_k, x_{ik}) and the target column (a_j) into a token sequence that is shared across all rows [34], [35].

We use a pretrained SLM g_θ as the corrector and fine-tune it by maximizing the conditional log-likelihood over cells not flagged as erroneous by the detector:

$$\ell(\theta) = \sum_{(i,j)\notin\mathcal{E}} \log p_\theta\big(x_{ij} \mid \mathbf{x}_i^{(j)}\big).$$

At inference time, for each detected erroneous cell $(i,j) \in \mathcal{E}$:

$$\hat{z}_{ij} = \arg\max_z \; p_\theta(z \mid \mathbf{x}_i^{(j)}),$$

and write $\hat{z}_{ij}$ back to obtain the repaired table.

This design delegates the expensive task of global structure discovery to the instructor, while the corrector provides efficient cell-level repair.

B. LasRepair+ method

Motivation. LasRepair selects a relevant context set for each target column, thereby reducing irrelevant information. However, column-level relevance alone cannot guarantee cell-level reliability, as the selected columns can still contain erroneous values from the dirty table. Using such low-quality values as context can mislead the corrector and consequently produce biased repairs. Addressing this issue requires going beyond selecting relevant context to selecting high-quality context [36]. We therefore extend LasRepair to LasRepair+, which adopts an EM-style procedure [29]–[31] that alternately refines the repaired table and updates the corrector.

Let $\theta^{(t)}$ denote the parameters of the corrector g_θ at iteration t, and let $\mathbf{D}_r^{(t)}$ denote the repaired table at the beginning of iteration t, with $\mathbf{D}_r^{(0)} = \mathbf{D}_e$. For each row i and target column j, we construct the input from current repaired table $D_r^{(t)}$:

$$x_i^{(j,t)} = \text{Serialize}\Big(\{(a_k, \mathbf{D}_r^{(t)}[i,k]) : k \in \mathcal{N}(j)\}, a_j\Big).$$

M-step: repaired context update. At iteration t, the current corrector $g_{\theta^{(t)}}$ induces a predictive distribution $p_{\theta^{(t)}}(z \mid x_i^{(j,t)})$ over candidate repairs. For each detected erroneous cell

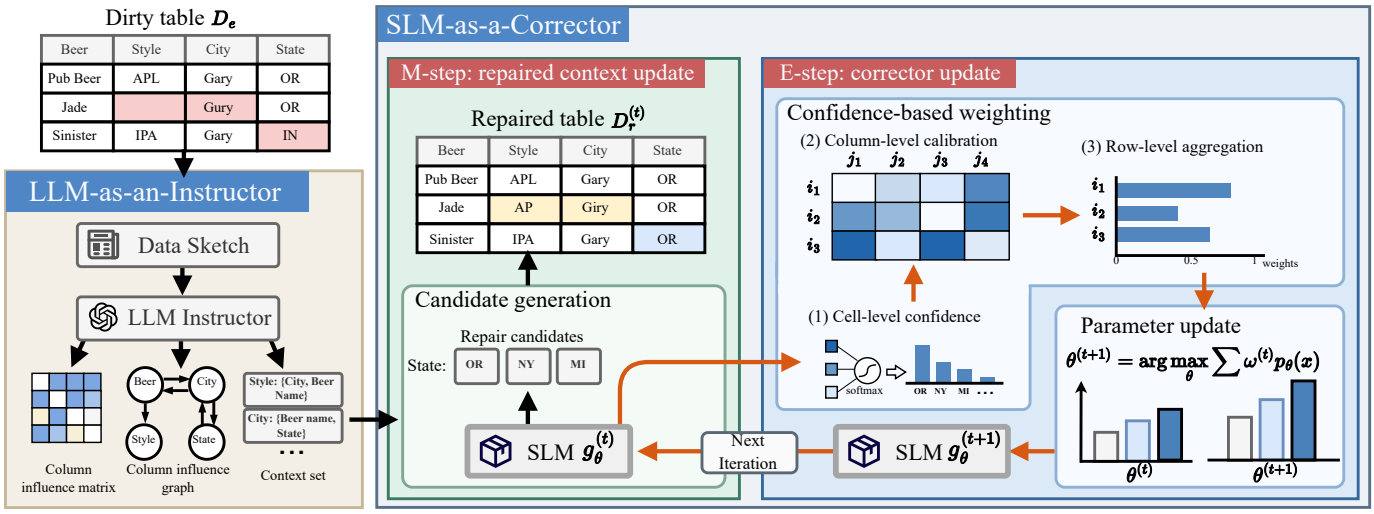

Fig. 3. Overview of LasRepair++. Starting from the dirty table, the LLM instructor constructs a column influence graph and extracts the target-specific context set as repair instructions. The SLM corrector then generates repair candidates and the cell-level confidence. A confidence matrix is then computed and aggregated into confidence-based weights, which are used to update the model parameters. The process terminates when the stop criterion is satisfied and outputs the final repaired table.

$(i, j) \in \mathcal{E}$, the corrector generates a repair by maximum a posteriori as follows:

$$\hat{z}_{ij}^{(t)} = \arg\max_{z} \; p_{\theta^{(t)}}(z \mid x_i^{(j,t)}).$$

We then construct the next repaired table $\mathbf{D}_r^{(t+1)}$ by replacing each detected erroneous cell with $\hat{z}_{ij}^{(t)}$. Since subsequent inputs are constructed from $\mathbf{D}_r^{(t+1)}$, this update turns generated repairs into an improved context for later repair steps.

E-step: corrector update. The corrector is then updated using the repaired context. Specifically, the parameters are updated by maximizing the conditional log-likelihood:

$$\theta^{(t+1)} = \arg\max_{\theta} \sum_{(i,j)\notin\mathcal{E}} \log p_\theta(\hat{z}_{ij}^{(t)} \mid x_i^{(j,t+1)}).$$

The update is performed using the repaired table $D_r^{(t+1)}$ as context, thereby adapting the corrector to progressively improved repair contexts.

Effect of iterative refinement. By alternating the above two update steps, LasRepair+ progressively refines the repaired table and the corrector parameters over successive iterations. Early iterations may still be affected by contaminated context, while later iterations benefit from an increasingly reliable repaired table and an updated corrector. This mutually reinforcing procedure directly addresses context contamination by making the selected context increasingly reliable across iterations. We further provide a theoretical monotonicity guarantee for the corresponding conditional log-likelihood.

Theorem 1 (Monotonicity of Likelihood)**.** *Let $\theta^{(t)}$ be the corrector parameter vector generated by the EM-style procedure at iteration t, and let $\ell(\theta)$ denote the corresponding conditional log-likelihood. Then, after each EM update, $\ell(\theta)$ is non-decreasing across iterations:*

$$\ell\big(\theta^{(t+1)}, z^{(t+1)}\big) \;\geq\; \ell\big(\theta^{(t)}, z^{(t)}\big) \qquad \textit{for all } t \geq 0.$$

A proof sketch is provided in Section 1.

C. LasRepair++ method

Motivation. The EM-style procedure in LasRepair+ improves the context quality and updates the corrector. However, the corrector still relies on stochastic optimization methods during training and probabilistic decoding during inference, both of which introduce uncertainty into the generated repairs. Such uncertainty can accumulate throughout the EM-style procedure, as generated repairs are repeatedly reused to construct contexts for subsequent iterations [32], [37]. Consequently, without appropriate regulation, unreliable repaired values increasingly bias later corrector updates and repair predictions [38]. Addressing this issue requires identifying uncertain repairs and limiting their contribution to corrector updates. We therefore propose LasRepair++, which introduces confidence-based weighting to filter low-confidence repair candidates and down-weight uncertain repaired rows. Specifically, confidence-based weighting addresses model uncertainty through three coordinated operations at the cell, column, and row levels:

(1) Cell-level confidence. Confidence-based weighting begins by quantifying the confidence of each generated repair at the cell level. Inspired by confident learning [39], we estimate the confidence by treating predicted repair values as classes. However, unlike the original confident learning formulation, treating each repair as a distinct class would result in a sparse class space, making confidence estimation unstable. We therefore use the predictive distribution at the first decoding position to estimate the cell-level confidence [40].

Let $\mathbf{Voc}$ denote the corrector vocabulary. At iteration t, for each detected erroneous cell $(i, j) \in \mathcal{E}$, the corrector produces a logit vector over $\mathbf{Voc}$ at the first decoding position. We extract the top-K first-token candidates:

$$A_{ij}^{(t)} = \{a_{ij(1)}^{(t)}, \ldots, a_{ij(K)}^{(t)}\},$$

together with their logits $\text{logit}_{ij(1)}^{(t)}, \ldots, \text{logit}_{ij(K)}^{(t)}$, where $a_{ij(r)}^{(t)}$ denotes the token with the r-th largest logit. We then

normalize these logits via a temperature-scaled softmax:

$$q_{ij(r)}^{(t)} = \frac{\exp(\text{logit}_{ij(r)}^{(t)}/\tau)}{\sum_{s=1}^{K}\exp(\text{logit}_{ij(s)}^{(t)}/\tau)}, \qquad \sum_{r=1}^{K} q_{ij(r)}^{(t)} = 1,$$

where $\tau > 0$ is the temperature parameter that controls the sharpness of the confidence distribution [41].

(2) Column-level calibration. We then calibrate the cell-level confidence using column-level information to reduce confidence bias. For a target column j and token v, let

$$\mathcal{I}_j^{(t)}(v) = \left\{ (i,r) \;\middle|\; (i,j) \in \mathcal{E},\ a_{ij(r)}^{(t)} = v, r \in [1,K] \right\}$$

be the set of positions whose first token is v. For a token v with $\mathcal{I}_j^{(t)}(v) \neq \emptyset$, we define the class-specific threshold as:

$$T_j^{(t)}(v) = \frac{1}{|\mathcal{I}_j^{(t)}(v)|} \sum_{(i,r)\in\mathcal{I}_j^{(t)}(v)} q_{ij(r)}^{(t)}.$$

Given the set of thresholds $\{T_j^{(t)}(v)\}_v$, for each detected erroneous cell $(i,j) \in \mathcal{E}$, we construct the corresponding accepted candidate index set as:

$$\mathcal{R}_{ij}^{(t)} = \{r \in [1,K] \mid q_{ij(r)}^{(t)} \geq T_j^{(t)}(a_{ij(r)}^{(t)})\}.$$

We then select the index of the most confident accepted token:

$$\hat{r}_{ij}^{(t)} = \arg\max_{r\in\mathcal{R}_{ij}^{(t)}} q_{ij(r)}^{(t)},$$

and use the corresponding first token to generate the remaining repair. If no candidate exceeds the threshold, we set $\hat{r}_{ij}^{(t)} = \varnothing$. This threshold calibrates confidence within each target column.

(3) Row-level aggregation. We next aggregate the calibrated confidence into row-level weights for corrector updates. For cells not flagged as erroneous, we set the confidence to 1. For accepted repairs, we use the probability of the selected first token as the confidence. If no candidate passes the threshold, we set the confidence to 0.

$$c_{ij}^{(t)} = \begin{cases} 1, & (i,j) \notin \mathcal{E}, \\ q_{ij(\hat{r}_{ij}^{(t)})}^{(t)}, & (i,j) \in \mathcal{E} \ \wedge\ \hat{r}_{ij}^{(t)} \neq \varnothing, \\ 0, & (i,j) \in \mathcal{E} \ \wedge\ \hat{r}_{ij}^{(t)} = \varnothing. \end{cases}$$

The calibrated confidence is then aggregated into a row-level reliability weight, denoted by $\omega_i^{(t)} = \frac{1}{d}\sum_{j=1}^{d} c_{ij}^{(t)}$, and let $\omega_i^{*(t)}$ be its normalized form. This aggregation assigns smaller weights to rows containing low-confidence or rejected repairs, thereby limiting their influence on subsequent updates.

After the confidence-based weighting, we refine the E-step to update the corrector more reliably:

$$\theta^{(t+1)} = \arg\max_{\theta} \sum_{(i,j)\notin\mathcal{E}} \omega_i^{*(t)} \log p_\theta\left(\hat{z}_{ij}^{(t)} \mid x_i^{(j,t)}\right).$$

By integrating confidence-based weighting in this way, LasRepair++ emphasizes training with more reliably generated repairs while down-weighting those affected by uncertainty. This reduces the propagation of model uncertainty during the EM-style procedure. We also state the theorem on the effectiveness of the confidence-based weighting.

Theorem 2 (Effectiveness of confidence-based weighting)**.** *Let the weighted gradient be $\hat{g}_\omega$, the unweighted gradient be $\hat{g}_u$, and the true gradient be g^*. We have*

$$\mathbb{E}\|\hat{g}_\omega(\theta) - g^*(\theta)\|^2 \leq \mathbb{E}\|\hat{g}_u(\theta) - g^*(\theta)\|^2.$$

This indicates that confidence-based weighting reduces the gradient variance induced by uncertainty.

A proof sketch is provided in Section IV-C.

D. Overall algorithm

We summarize the complete workflow of LasRepair++ in Algorithm 1. Given a dirty table $\mathbf{D}_e$ and a detected error set $\mathcal{E}$, LasRepair++ first invokes the LLM as an instructor to compute a score matrix $\mathbf{W}$ and constructs a directed weighted graph $\mathcal{G}$ based on $\mathbf{W}$. It then derives a context set $\mathcal{N}(j)$ for each target column j (Lines 2–5).

After the LLM-as-an-instructor stage, LasRepair++ performs the EM-style procedure in Lines 6–23. Each iteration first executes the M-step, where the corrector generates a repaired value for each detected erroneous cell under the selected context to update the repaired table $\mathbf{D}_r^{(t+1)}$ (Lines 7–11). It then executes the E-step, which incorporates confidence-based weighting to update the corrector (Lines 12–22). Specifically, the algorithm first estimates cell-level confidence over the top-K first-token candidates (Lines 12–14). It then performs column-level calibration by computing confidence thresholds and selecting the accepted first token (Lines 15–17). Next, the calibrated cell-level confidence is aggregated into row-level weights (Lines 18–21). Finally, these confidence-based weights are incorporated into the corrector update (Line 22). The algorithm terminates when the confidence-based weights converge or the maximum number of iterations is reached, and returns the final repaired table $\mathbf{D}_r^{(t)}$.

IV. ANALYSIS

A. Time complexity analysis

We analyze the time complexity of LasRepair++ by separating the instructor stage and the corrector stage. Since the instructor is invoked once on a compact sketch, we exclude external LLM inference from the asymptotic complexity and focus on the computation performed by the corrector.

After obtaining the score matrix $\mathbf{W}$, for each target column $j \in \{1,\ldots,d\}$, we construct an h-hop influence tree and select the context set via the threshold ϵ. A straightforward dynamic programming implementation costs $O(hd^2)$ per target column, yielding a total cost of $O(hd^3)$ for graph construction.

For the EM-style procedure, let $|\mathcal{E}|$ be the number of detected erroneous cells, and let C_{SLM} denote the average per-cell cost of one corrector pass on a serialized input. At each iteration of the EM-style procedure, the corrector generates repairs and confidence for cells in $\mathcal{E}$, and is then fine-tuned using the confidence-based weights. Therefore, for e fine-tuning epochs over T iterations, the dominant cost is

Algorithm 1: Overall Workflow of LasRepair++

Input: dirty table $\mathbf{D}_e$, detected error set $\mathcal{E}$, LLM $\mathcal{M}$, SLM $g_{\theta^{(0)}}$, maximum iterations T, hop budget h, top-K size k, context threshold ϵ, convergence tolerance η, temperature τ.
Output: repaired table $\mathbf{D}_r$.

$\mathbf{D}_r^{(0)} \leftarrow \mathbf{D}_e$
// **Stage I: LLM as an instructor**
$\mathbf{W} \leftarrow \textsc{QueryLLM}(\mathcal{M}, \textsc{Sample}(\mathbf{D}_e))$
$\mathcal{G} \leftarrow \textsc{WeightGraph}(\mathbf{W})$
for $j \in \mathcal{C}$ **do**
  $\mathcal{N}(j) \leftarrow \textsc{KHopTreeNeighbour}(\mathcal{G}, h, j, \epsilon)$
// **Stage II: EM-style procedure**
while $t \le T$ ***and*** $\Delta_\omega \ge \eta$ **do**
  // **M-step: repaired context update**
  $\mathbf{D}_r^{(t+1)} \leftarrow \mathbf{D}_r^{(t)}$
  for $(i,j) \in \mathcal{E}$ **do**
    $\mathbf{x}_i^{(j,t)} \leftarrow \textsc{Serialize}(\mathbf{D}_r^{(t)}, i, \mathcal{N}(j))$
    $z_{ij}^{(t)} \leftarrow \arg\max_z p_{\theta^{(t)}}(z \mid x_i^{(j,t)})$
    $\mathbf{D}_r^{(t+1)}[i,j] \leftarrow z_{ij}^{(t)}$
  // **E-step: corrector update**
  // **Cell-level confidence**
  for $(i,j) \in \mathcal{E}$ **do**
    $(A_{ij}^{(t)}, \text{logit}_{ij}^{(t)}) \leftarrow \textsc{TopKToken}(g_\theta^{(t)}, \mathbf{x}_i^{(j,t)}, k)$
    $q_{ij}^{(t)} \leftarrow \textsc{Softmax}(\text{logit}_{ij}^{(t)}, \tau)$
  // **Column-level calibration**
  for $j \in \mathcal{C}$ **do**
    $T_j^{(t)}(\cdot) \leftarrow \textsc{TokThreshold}(A_{ij}^{(t)}, q_{ij}^{(t)} : (i,j) \in \mathcal{E})$
    $\hat{r}_{ij}^{(t)} \leftarrow \textsc{SelectTok}\big(A_{ij}^{(t)}, q_{ij}^{(t)}, T_j^{(t)}(\cdot)\big)$
  // **Row-level aggregation**
  for *all* (i,j) **do**
    $c_{ij}^{(t)} \leftarrow \textsc{Confidence}(q_{ij}^{(t)}, \hat{r}_{ij}^{(t)})$
  $\omega^{(t)} \leftarrow \textsc{Aggregation}(c^{(t)})$
  $\omega^{*(t)} \leftarrow \textsc{Normalize}(\omega^{(t)})$
  $\theta^{(t+1)} \leftarrow \textsc{WeightUpdate}(\theta^{(t)}, \omega^{*(t)})$
  $t \leftarrow t+1$
return $\mathbf{D}_r^{(t)}$

$O(T \cdot e \cdot |\mathcal{E}| \cdot C_{\text{SLM}})$. The additional cost of computing class-specific thresholds and confidence-based weights is linear in the number of candidates and detected erroneous cells, and is dominated by corrector inference and training in practice.

B. Effectiveness of the EM-style procedure

We now justify the monotonicity claim in Theorem 1 under the idealized setting where the repaired context update and the corrector update are solved exactly. Let $\{(\theta^{(t)}, \hat{\mathbf{z}}^{(t)})\}_{t\ge 0}$ be the sequence generated by the updates in Section III-B, where $\hat{\mathbf{z}}^{(t)} = \{\hat{z}_{ij}^{(t)}\}_{(i,j)\in\mathcal{E}}$. For a fixed set of serialized repaired inputs, define the conditional log-likelihood as

$$\ell(\theta, \hat{\mathbf{z}}) = \sum_{(i,j)\in\mathcal{E}} \log p_\theta(\hat{z}_{ij} \mid x_i^{(j)}).$$

Proof sketch of Theorem 1. The argument follows from the properties of the alternating optimization procedure.

Repaired context update. Given $\theta^{(t)}$, the repaired context update selects the most likely repair value for each detected erroneous cell. Therefore, under the exact update assumption,

$$\ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t+1)}) \ge \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t)}).$$

Corrector update. Given $\hat{\mathbf{z}}^{(t+1)}$, the corrector update optimizes the same objective with respect to θ. Hence,

$$\ell(\theta^{(t+1)}, \hat{\mathbf{z}}^{(t+1)}) \ge \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t+1)}).$$

Combining the two inequalities yields

$$\ell(\theta^{(t+1)}, \hat{\mathbf{z}}^{(t+1)}) \ge \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t+1)}) \ge \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t)}),$$

which proves the monotonic non-decrease of the conditional log-likelihood under the idealized EM-style assumptions.

For LasRepair++, the confidence-based weights are computed before the weighted corrector update and then fixed during that update. Therefore, the same alternating-optimization argument applies to the corresponding weighted objective (the confidence-based weighting). The complete proof is provided in our supplementary repository [42].

C. Effectiveness of confidence-based weighting

We now theoretically analyze the effect of the confidence-based weighting in LasRepair++. As introduced in Section III-C, LasRepair++ aggregates cell-level confidence into confidence-based weights and uses the normalized weights ω_i^* to reduce the gradient variance induced by uncertainty. We now justify that claim for Theorem 2. Consider one fixed iteration and omit the superscript t.

Proof sketch of Theorem 2. For each row i, define the loss

$$\hat{\ell}_i(\theta) = \sum_{(i,j)\in\mathcal{E}} -\log p_\theta(\hat{z}_{ij} \mid x_i^{(j)}),$$

and the ground-truth loss

$$\ell_i^*(\theta) = \sum_{(i,j)\in\mathcal{E}} -\log p_\theta(z_{ij} \mid x_i^{(j)}).$$

Let $\hat{g}_i(\theta) = \nabla_\theta \hat{\ell}_i(\theta)$ and $g_i^*(\theta) = \nabla_\theta \ell_i^*(\theta)$ denote the gradients computed from generated repairs and ground-truth values, respectively. Assume that

$$\hat{g}_i(\theta) = g_i^*(\theta) + \epsilon_i, \qquad \epsilon_i \sim N(0, \sigma^2/\omega_i^*),$$

where $\{\epsilon_i\}_{i=1}^n$ are conditionally independent. For the gradients in the Theorem 2, we have $\hat{g}_\omega(\theta) = \sum_{i=1}^n \omega_i^* \hat{g}_i(\theta)$, $\hat{g}_u(\theta) = \frac{1}{n}\sum_{i=1}^n \hat{g}_i(\theta)$ and $g^*(\theta) = \frac{1}{n}\sum_{i=1}^n g_i^*(\theta)$.

Under this setting, rows with higher confidence induce lower gradient variance. Therefore, the weighted estimator assigns less weight to uncertain rows than does uniform averaging. A direct variance comparison, together with the Cauchy–Schwarz inequality, yields

$$\mathbb{E}\|\hat{g}_\omega(\theta) - g^*(\theta)\|^2 \le \mathbb{E}\|\hat{g}_u(\theta) - g^*(\theta)\|^2.$$

This proves the claim in Theorem 2. The full proof is provided in our supplementary repository [42].

TABLE II
Statistics of the datasets. R-rate and C-rate are Row-level error rate and Cell-level error rate, respectively.

Datasets	#Tuples	#Attrs	R-rate, %	C-rate, %	Error type
Hospital	1,000	20	40.70	2.67	T, VAD
Flight	2,376	7	80.13	34.51	MV, FI, VAD
Beers	2,410	11	100.00	13.93	MV, FI, VAD
Walmart	4,653	5	62.91	17.92	SY
Tax_20k	20,000	15	4.76	0.32	T, FI, VAD
Shuttle	43,500	10	73.55	12.44	SY
Tax_200k	200,000	15	1.46	0.10	T, FI, VAD

V. Experiments

A. Dataset description

We evaluate LasRepair++ on 7 benchmark datasets widely used in prior data-cleaning studies [6], [9], [17]. Table II summarizes their statistics, including dataset size, error types, cell-level error rates, and row-level error rates. The error types include missing value (MV), typo (T), violated attribute dependency (VAD), and formatting issue (FI).

Synthetic error generation. For datasets marked as SY, only a clean version is available. Therefore, we follow the prior work [43] to inject errors and construct the dirty version. The injection process follows the Missing at Random (MAR) assumption [44], where corruption depends on observed information but not the unknown clean value. Specifically, we first sample cells according to a target error rate. For each selected cell, we sample an error type based on the column data type and apply the corresponding operator. For MV, we replace the value with an empty token. For T, we apply 1–k character-level edits to strings, where $k \in \{1, 2, 3\}$ and $k \leq |x_{ij}|/2$. Each edit is uniformly sampled from insertion, deletion, substitution, and transposition. For numerical columns, VAD adds zero-mean noise scaled by column statistics, while FI applies format transformations such as integer-to-float conversion.

B. Experimental setup

Baselines. We compare LasRepair++ with 8 representative baselines, covering both non-generative and generative data repair methods. For compact presentation, we group the conventional and discriminative baselines as non-generative methods in the experimental tables: 1) Baran [17], a data-driven repair framework that generates candidate repairs and selects corrections using learned classifiers; 2) HoloClean [12], which combines integrity constraints, external signals, and statistical inference to identify likely correct values; 3) BoostClean [45], which selects cleaning operations to improve downstream data quality; 4) Unified [13], which performs tolerant repair under functional dependencies using a minimum description length principle; 5) BigDansing [8], which accelerates rule-based data repair with two optimization strategies; 6) Holistic [7], which models equivalent classes and conflicts via a hypergraph for data repair. The generative baselines include: 7) Jellyfish [19], an LLM-based model for data preprocessing and repair; 8) GIDCL [18], a graph-enhanced LLM-based data cleaning framework.

Metrics. We evaluate repair quality using three complementary metrics: F1-score, Error Drop Rate (EDR), and Record Distance Reduction Rate (RDRR). F1-score [46] is the primary metric and is computed from exact match repair outcomes following prior work. To further characterize repair performance, we additionally report EDR [6] and RDRR, which capture complementary aspects of repair quality.

EDR measures the relative reduction in the number of erroneous cells after repair:

$$\text{EDR} = \frac{\text{OEC} - \text{REC}}{\text{OEC}},$$

where OEC and REC denote the *original error count* and *remaining error count*, respectively. A larger EDR indicates that more original errors are removed.

To evaluate partial repairs that do not exactly match the ground truth, we further introduce RDRR based on the Levenshtein edit distance d_{lev} [47]. Let g_i denote the ground-truth value and r_i denote the corresponding value in either the dirty or repaired dataset. Let N denote the number of evaluated cells. We first compute the average normalized edit distance:

$$\text{AED} = \frac{1}{N} \sum_{i=1}^{N} \frac{d_{\text{lev}}(g_i, r_i)}{\max(|g_i|, |r_i|)},$$

RDRR is then defined as

$$\text{RDRR} = \frac{\text{AED}_d - \text{AED}_r}{\text{AED}_d},$$

where the subscripts d and r denote the dirty and repaired datasets, respectively. RDRR measures the relative reduction in edit distance achieved by repair. It is particularly useful when a method improves a corrupted value substantially but does not recover the exact ground truth. Higher values of all three metrics indicate better performance.

Implementation details. All baselines are executed with their default hyperparameters when available. We use *Jellyfish-7B* in the Jellyfish and GIDCL experiments. For LasRepair++, we use *T5-large* [48], with 770M parameters, as the SLM corrector, and GPT-5 [49] as the instructor. Unless otherwise specified, we train the corrector for 3 epochs, run at most 10 EM-style iterations, and set the softmax temperature to $\tau = 1$. For large-scale datasets, we sample 2,000 records to fine-tune the corrector while evaluating repair quality on the target cells. All experiments are conducted on a server with an Intel Xeon Silver 4313 CPU and an NVIDIA RTX A5000 GPU.

C. Effectiveness evaluation

Exp-1: F1-score, EDR, and RDRR. We first evaluate the effectiveness of LasRepair++ against 8 representative baselines on 7 benchmark datasets. Table III reports the F1-score, EDR, and RDRR of all methods across the benchmark datasets. Some non-generative methods fail to finish on large datasets within the time or memory budget. These cases are marked as OOT (out of time) or OOM (out of memory).

Overall, LasRepair++ achieves the best F1-score on all 7 datasets. Compared with the dataset-wise strongest baseline, LasRepair++ improves the average F1-score from 0.6882 to 0.8130, yielding a relative gain of 18.1%. This demonstrates

TABLE III

Overall repair performance on 7 benchmark datasets. We compare non-generative and generative repair methods using F1, EDR, and RDRR, where higher values indicate better quality. The best result is highlighted in bold and underlined, and the second-best result is highlighted in bold. The last block reports average improvement of LasRepair++ over each baseline. OOM and OOT denote out-of-memory and out-of-time failures.

Datasets	Metrics	Non-generative						Generative		
		Baran	HoloClean	BoostClean	Unified	BigDansing	Holistic	Jellyfish	GIDCL	LasRepair++
Hospital	F1	0.5844	0.6262	0.3312	0.7825	0.6210	0.6101	0.8871	**0.8925**	**0.9632**
	EDR	0.4165	0.4538	0.3132	0.7612	-0.0716	-0.0187	0.8730	**0.8822**	**0.9871**
	RDRR	0.3548	0.4432	0.1037	0.5960	0.0245	0.0449	0.8014	**0.8256**	**0.8647**
Flights	F1	**0.6369**	0.4734	0.0620	0.5075	0.1090	0.2313	0.6324	0.5903	**0.8798**
	EDR	0.4913	-0.1339	-0.0028	0.0541	-0.3543	-0.1423	**0.5919**	0.4872	**0.9072**
	RDRR	0.5315	-0.2341	-0.0039	0.0411	-1.0856	-0.7324	**0.5830**	0.3156	**0.9683**
Beers	F1	0.7576	0.0467	0.0109	0.0324	0.1157	0.0892	**0.7832**	0.4332	**0.8894**
	EDR	0.7082	-3.6209	-0.7192	-0.2442	-0.0108	-0.0093	**0.7264**	-0.1335	**0.8838**
	RDRR	**0.7842**	-5.7141	-0.9923	-0.3121	-0.1192	-0.1372	0.7197	-0.2290	**0.9253**
Walmart	F1	0.2460	**0.5315**	0.3167	0.3528	0.3797	0.2912	0.4712	0.4372	**0.7409**
	EDR	0.0368	0.0276	0.1049	0.3006	-0.2334	-0.3876	**0.3122**	-0.0897	**0.5317**
	RDRR	0.0113	0.0198	0.1953	**0.4414**	-0.5298	-0.4893	0.3464	-0.2315	**0.5417**
Tax_20k	F1	0.3512	0.0000	0.0000	OOT	0.2413	0.2413	0.5774	**0.5923**	**0.6944**
	EDR	0.2490	-0.0198	0.0000	–	-0.9127	-0.9127	**0.5008**	0.4863	**0.6793**
	RDRR	0.1738	-0.6295	-0.3957	–	-0.3531	-0.3531	0.6022	**0.6139**	**0.7129**
Shuttle	F1	0.7775	OOM	0.0000	0.0000	OOT	OOT	**0.8359**	0.7209	**0.8898**
	EDR	0.6931	–	-0.4932	-0.3315	–	–	**0.8142**	0.6559	**0.7652**
	RDRR	0.7032	–	-0.6587	-0.5114	–	–	**0.8462**	0.7621	**0.9708**
Tax_200k	F1	0.3276	OOM	0.0000	OOT	OOT	OOT	0.5227	**0.5450**	**0.6332**
	EDR	0.1814	–	0.0000	–	–	–	0.4432	**0.4573**	**0.6372**
	RDRR	0.2321	–	-0.4498	–	–	–	0.5486	**0.5686**	**0.7567**
Average Improvement	F1	0.2866	0.5728	0.7095	0.5732	0.6030	0.6035	0.1397	0.2109	–
	EDR	0.3776	1.2446	0.8880	0.6970	1.0003	0.9843	0.1654	0.3819	–
	RDRR	0.4213	1.6935	1.1345	0.7836	1.1148	1.0582	0.1847	0.4450	–

that the proposed instructor–corrector design, EM-style procedure, and confidence-based weighting jointly improve repair accuracy. Among the non-generative methods, Baran is generally the strongest baseline. However, its performance varies substantially across datasets, especially when the candidate generation is insufficient. Among the generative methods, Jellyfish and GIDCL typically outperform most non-generative baselines, demonstrating the benefits of LLM-based repair. Even against these strong generative competitors, LasRepair++ remains consistently superior, achieving an average F1-score of 0.8130. This indicates that directly relying on large generative models is less effective than using an LLM as an instructor and an SLM as a corrector for repair.

The same trend is reflected for EDR and RDRR. Compared with the dataset-wise strongest baselines, LasRepair++ improves the average EDR from 0.6121 to 0.7702 and the average RDRR from 0.6661 to 0.8201. LasRepair++ obtains the highest EDR on 6 out of the 7 datasets and the highest RDRR on all datasets. This suggests that LasRepair++ not only exactly corrects more erroneous cells, but also moves incorrect values closer to the ground truth.

Exp-2: Robustness under varying error rates. We next evaluate whether LasRepair++ remains effective when the input dataset becomes increasingly contaminated. We select Flight and Hospital for this experiment. These two datasets cover all error types considered in our evaluation and are moderately sized, enabling controlled analysis across different error rates. Specifically, we vary the injected error rate from 1% to 50%, and compare LasRepair++ with representative baselines from both the non-generative and generative categories.

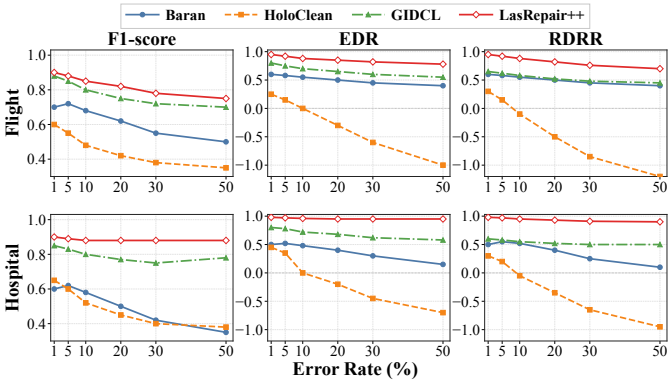

Fig. 4. Repair performance under varying cell-level error rates on Flight (a–c) and Hospital (d–f), evaluated by F1-score, EDR, and RDRR.

Figure 4 reports F1-score, EDR, and RDRR under different error rates. As the error rate increases, the performance of all methods degrades. By comparison, LasRepair++ degrades more slowly and consistently achieves the best performance across all three metrics on both datasets. At the highest error rate of 50%, LasRepair++ still outperforms the strongest baseline, GIDCL, by 7.22% and 19.53% in F1-score on the Flight and Hospital datasets, respectively. These results indicate that LasRepair++ is more robust to severe corruption and remains effective even when the input dataset is highly contaminated.

Exp-3: Performance under different numbers of iterations. We further study the effect of the EM-style procedure. We run LasRepair++ with different numbers of iterations on Beers, Hospital, Flight, and Walmart, and report F1-score, EDR,

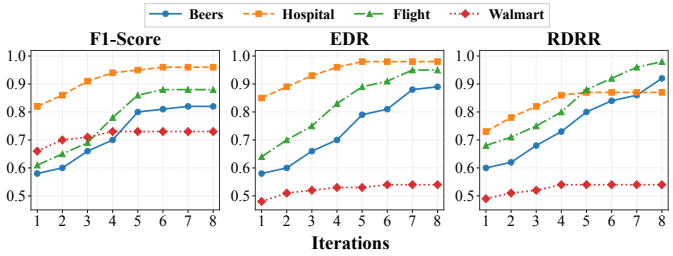

Fig. 5. Performance under different numbers of iterations on datasets Beers, Hospital, Flight, and Walmart. We report F1-score, EDR, and RDRR in (a)–(c), respectively.

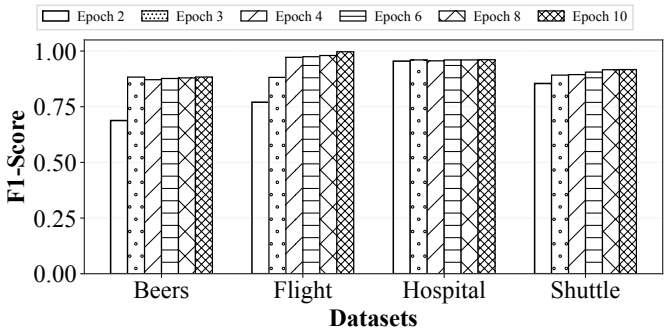

Fig. 6. Impact of fine-tuning epochs on repair performance across datasets. F1-score improves noticeably from 2 to 3 epochs, while gains become marginal after about 6 epochs.

and RDRR in Figure 5. These datasets exhibit diverse repair difficulties, providing a representative view of how repair quality evolves across iterations. The results show that repair performance generally improves as the number of iterations increases. The average F1-score increases from 0.6689 at the first iteration to 0.8662 at the final iteration, yielding a relative improvement of 29.50%. Similar trends are observed for EDR and RDRR, which improve by approximately 30.6% and 31.7% on average across the 4 datasets.

These results confirm that updating the repaired dataset and fine-tuning the corrector with generated repairs can progressively improve the quality of repairs. The improvement is sometimes stepwise rather than smooth, because the corrector may learn a recurring repair pattern after several iterations and then correct many similar cells. The marginal gain decreases in later iterations, and most improvements are achieved within the first 6–7 rounds. We therefore set the maximum number of iterations to $T = 10$ in the remaining experiments to balance between effectiveness and computational cost.

D. Parameter analysis

Exp-4: Varying the number of epochs. We study the sensitivity of LasRepair++ to the fine-tuning budget of the corrector. We select 4 datasets spanning different data scales, allowing us to examine whether the effect of the training budget remains consistent. Specifically, we vary the number of fine-tuning epochs over $2, 3, 4, 6, 8, 10$ and report the resulting F1-scores in Figure 6.

Overall, increasing the epoch number from 2 to 3 yields clear improvements across datasets. This indicates that a very small training budget is insufficient for the corrector to adapt to dataset-specific repair patterns. Increasing the epoch number from 2 to 3 improves the average F1-score from 0.8169 to 0.9043, while further increasing it to 10 epochs only raises the average F1-score to 0.9397. This suggests diminishing returns from longer training. We therefore use 3 epochs as the default setting in the remaining experiments, which provides a good balance between accuracy and computational cost.

TABLE IV
Effect of the softmax temperature (τ) on repair performance.

Dataset	0.5			1.0			2.0		
	F1	EDR	RDRR	F1	EDR	RDRR	F1	EDR	RDRR
Beers	0.8721	0.8697	0.9147	0.8831	0.8807	0.9262	0.8554	0.8531	0.8971
Shuttle	0.8591	0.8507	0.8913	0.8920	0.8833	0.9254	0.8701	0.8616	0.9027

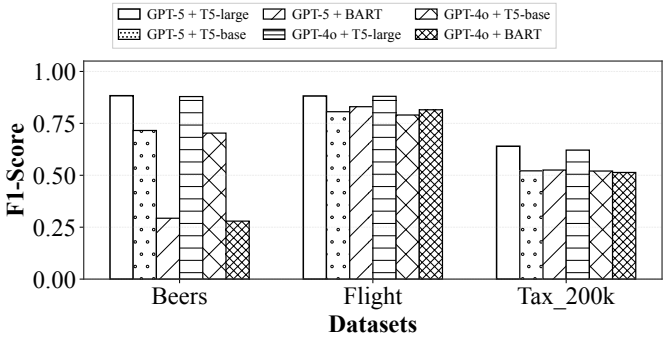

Fig. 7. F1-score under different LLM–SLM combinations on Beers, Flight, and Tax_200k. Larger correctors consistently yield better repair accuracy, while changing the instructor has a smaller effect.

Exp-5: Varying temperature in confidence-based weighting. We next evaluate the effect of the softmax temperature τ in the confidence-based weighting. We vary τ in $\{0.5, 1.0, 2.0\}$ and report F1-score, EDR, and RDRR on 2 representative datasets with substantially different scales, as summarized in Table IV. Overall, $\tau = 1.0$ achieves the best overall performance on both datasets across all 3 metrics, indicating that a moderate temperature yields the most effective confidence-based weighting. Compared with $\tau = 0.5$ and $\tau = 2.0$, $\tau = 1.0$ improves the average F1-score by 2.54% and 2.87%, respectively. When τ is too small, the confidence distribution becomes overly sharp, causing the model to place excessive weight on a small number of candidates, which may amplify overconfident but incorrect repairs. When τ is too large, the distribution becomes overly smooth, weakening the distinction between reliable and unreliable repairs. We therefore use $\tau = 1.0$ as the default setting.

Exp-6: Effect of LLM–SLM combinations. We investigate how different instructor and corrector choices affect repair performance. For the LLM instructor, we compare GPT-5 and GPT-4o [50]. For the SLM corrector, we vary model capacity by choosing from T5-large (0.77B parameters), T5-base (0.22B), and BART-base (0.1B) [51]. We select the Beers, Flight, and Tax_200k datasets because they vary in scale and include all error types. Figure 7 reports the resulting F1-scores.

The results show that the capacity of the corrector has

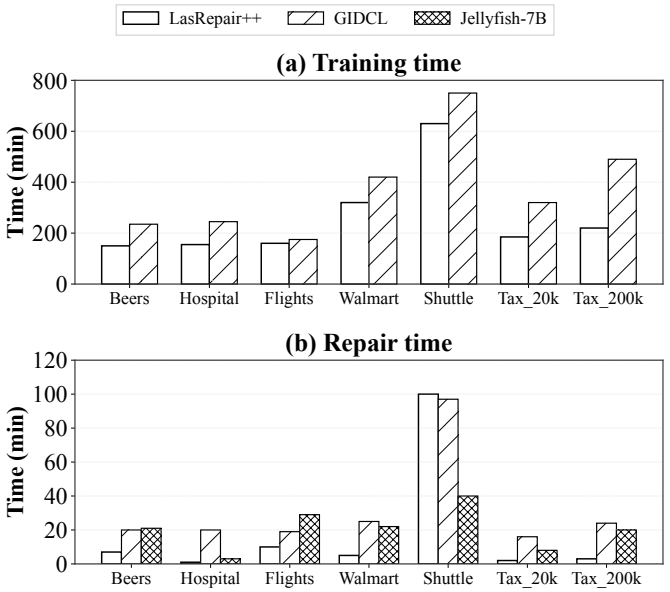

Fig. 8. Running time of different repair methods on seven datasets. Panels (a) and (b) report training time and repair time, respectively, both in minutes.

a much stronger impact than the choice of the instructor. With GPT-5 as the instructor, replacing T5-large with T5-base and BART-base reduces the average F1-score by 15.04% and 31.45%. In contrast, replacing GPT-5 with GPT-4o while retaining T5-large reduces the average F1-score by 1.01%. These results indicate that the corrector capability has a substantially greater impact on repair performance than the instructor. This trend is especially evident on more challenging datasets, where effective repair requires the corrector to learn complex value patterns and dependencies. In contrast, when the corrector is fixed, changing the instructor from GPT-5 to GPT-4o leads to minor differences. This suggests that once the instructor provides a reasonable column-level context structure, the dominant factor becomes the repair capacity of the corrector. We therefore use GPT-5 as the instructor and T5-large as the corrector.

Exp-7: Effect of training set size. We evaluate the sample efficiency of LasRepair++ on large-scale datasets. We vary the number of sampled training tuples on two large-scale datasets, Shuttle and Tax_200k. Specifically, we train with $\{200, 500, 1{,}000, 2{,}000, 3{,}000\}$ tuples and report F1-score, EDR, and RDRR in Table V.

Overall, performance improves as the training set size increases. With 200 tuples, the corrector is exposed to limited dataset-specific training data and therefore achieves relatively low accuracy. Increasing the training set size to 1,000 tuples brings substantial gains, showing that LasRepair++ can learn useful repair behavior from a moderate number of examples. Further increasing the training set size to 2,000–3,000 tuples provides additional but smaller improvements, suggesting diminishing returns. This result supports our default choice of using 2,000 sampled tuples for large-scale datasets, which balances repair performance and cost.

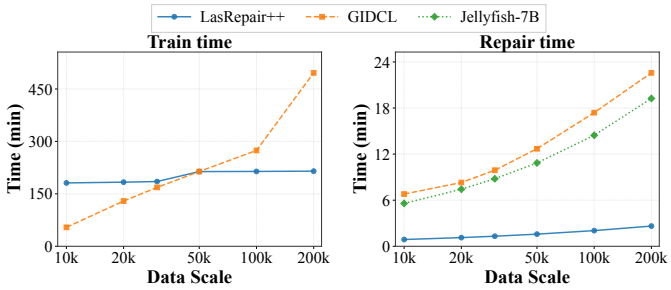

Fig. 9. Scalability of different repair methods under increasing data scales on the Tax dataset.

E. Efficiency evaluation

Exp-8: Runtime. We compare the runtime of LasRepair++ against the generative repair baselines, reporting both training time and repair time on 7 datasets in Figure 8.

In the training stage, LasRepair++ fine-tunes a lightweight corrector rather than an LLM. Its training time is lower than or comparable to the generative baselines on most datasets. Compared with GIDCL, LasRepair++ reduces training time by approximately 40% on average across the evaluated datasets.

In the repair stage, LasRepair++ is also consistently efficient because the instructor is used only once to select column-level context, while the corrector performs cell-level repair. Therefore, the repair cost scales mainly with the number of detected erroneous cells and the corrector inference cost. Across the seven datasets, LasRepair++ reduces repair time by 74% on average compared with GIDCL and by about 69% compared with Jellyfish. The exception is Shuttle, where the high error rate results in a large number of detected cells and thus incurs repeated repair operations. Overall, LasRepair++ achieves an efficiency–accuracy trade-off.

Exp-9: Scalability. Figure 9 reports the training and repair time of LasRepair++, Jellyfish-7B, and GIDCL as the Tax dataset scales from 10k to 200k tuples. Jellyfish-7B is omitted from the training comparison because it does not require dataset-specific fine-tuning.

For training, LasRepair++ exhibits a relatively flat growth trend because it fine-tunes the corrector on a bounded sample of tuples for large-scale datasets. When the dataset size increases by $20\times$, its training time increases only from 181.26 to 214.91 minutes, corresponding to $1.19\times$ the original time. In contrast, the training time of GIDCL grows by $9.15\times$, from 54.19 to 496.13 minutes.

For repair, GIDCL and Jellyfish-7B exhibit similar trends because they use the same underlying LLM. As the dataset size increases from 10k to 200k, the repair time of LasRepair++ grows by approximately $2.9\times$, whereas GIDCL and Jellyfish-7B grow by about $3.3\times$ and $3.4\times$, respectively. These results demonstrate the superior scalability of LasRepair++.

F. Ablation study

Exp-10: Ablation study. We conduct an ablation study to quantify the contribution of three core components in LasRepair++: (i) the LLM as an instructor for context selection, (ii)

TABLE V
Effect of training size on repair performance, evaluated by F1-score, EDR, and RDRR.

Dataset\Training size	200			500			1000			2000			3000		
	F1	EDR	RDRR	F1	EDR	RDRR	F1	EDR	RDRR	F1	EDR	RDRR	F1	EDR	RDRR
Tax_200k	0.3458	0.3474	0.4052	0.5174	0.5198	0.6063	0.6172	0.6201	0.7232	0.6398	0.6428	0.7497	0.6284	0.6313	0.7363
Shuttle	0.5580	0.5525	0.5789	0.5775	0.5718	0.5991	0.7947	0.7869	0.8245	0.8745	0.8659	0.9073	0.8869	0.8782	0.9201

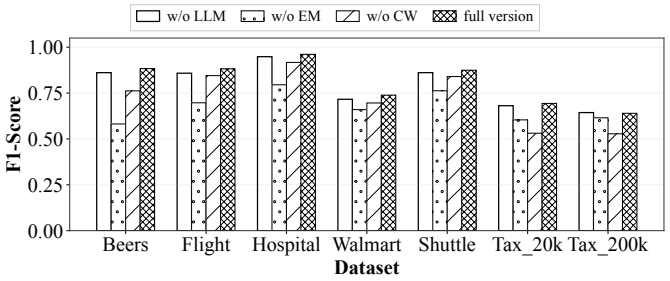

Fig. 10. Ablation study of LasRepair++ on seven benchmark datasets. We individually remove each component, including LLM-based context selection (LLM), the EM-style procedure (EM), and confidence-based weighting (CW), and report the resulting F1-scores.

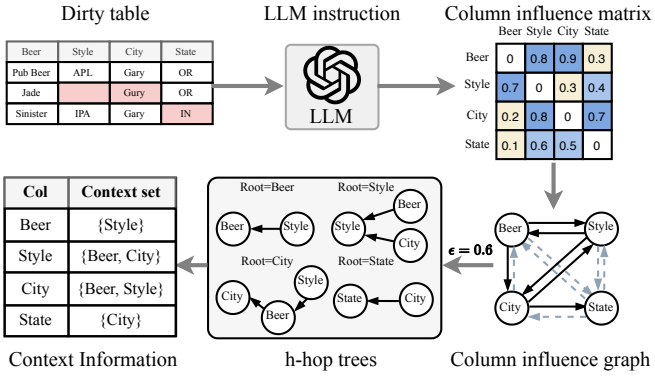

Fig. 11. Case study demonstrating how the column-specific context sets are extracted from a dirty dataset.

the EM-style procedure, and (iii) the confidence-based weighting. For each ablated variant, we remove one component while keeping the remaining settings unchanged. Figure 10 reports the F1-scores on 7 benchmark datasets.

Removing the EM-style procedure causes the largest degradation: the average F1-score drops from 0.8104 to 0.6736, with especially large declines on Beers and Flight. Removing confidence-based weighting also reduces the average F1-score to 0.7314, with the largest drops on Tax_20k and Tax_200k. Removing the instructor results in an average F1-score decrease of 1.81%, consistent with its primary role in filtering irrelevant attributes to improve inference efficiency. Overall, the EM-style procedure makes the largest contribution, followed by confidence-based weighting and the instructor. Taken together, the ablation results show that the three components of LasRepair++ provide complementary benefits.

G. Case study

We present a case study to illustrate the column-specific context selection of the LLM instructor. As shown in Figure 11, the instructor estimates inter-column influence from a simplified Beers dataset and extracts a context set for each target column from the resulting graph. For example, although Style has no direct influence on City under the threshold $\epsilon = 0.6$, its influence propagates through Beer. Therefore, both columns are selected as contexts for repairing City. This example demonstrates that the instructor can extract compact and relevant contexts to support repair.

VI. Related Work

Existing methods can be broadly divided into *conventional*, *discriminative*, and *generative* methods.

Conventional methods rely on integrity constraints or probabilistic inference. Constraint-oriented methods such as Holistic [7] and BigDansing [8] repair violations of integrity constraints, with BigDansing emphasizing scalability. Unified [13] formulates repair under functional dependencies using a minimum-description-length principle. HoloClean [12] combines constraints, external knowledge, and statistical dependencies through probabilistic inference.

Discriminative methods learn predictive models to select appropriate repairs. Baran [17] generates candidates from contextual representations and uses column-specific classifiers for selection. BoostClean [45] learns combinations of detection and repair operations to improve downstream prediction.

Generative methods leverage pretrained language models to produce repairs from serialized table contexts. Jellyfish [19] adapts local LLMs for general-purpose data preprocessing tasks, including repair. GIDCL [18] combines graph-based table modeling with LLM-based repair. Although these methods improve repair capability, they still depend on context quality and suffer from uncertain model outputs. This limitation motivates our focus on instructor-guided context selection, EM-style procedure, and confidence-based weighting.

VII. Conclusion

In this paper, we present the LasRepair family, which combines the semantic reasoning capability of LLMs with the efficiency of SLMs for scalable data repair. The base framework, LasRepair, uses an LLM as a table-level structural instructor to select compact and relevant repair contexts, and fine-tunes an SLM as a scalable cell-level corrector. Building on this, LasRepair+ adopts an EM-style procedure to jointly refine the repaired table and the corrector over successive iterations. LasRepair++ further applies confidence-based weighting to reduce the influence of unreliable generated repairs. Experiments on 7 benchmark datasets highlight the effectiveness and efficiency of our method.

REFERENCES

[1] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "Cleanml: A study for evaluating the impact of data cleaning on ml classification tasks," 2021. [Online]. Available: https://arxiv.org/abs/1904.09483

[2] J. Wang, K. Wang, Y. Zhang, W. Zhang, X. Xu, and X. Lin, "On llm-enhanced mixed-type data imputation with high-order message passing," 2025. [Online]. Available: https://arxiv.org/abs/2501.02191

[3] J. Li, A. Fang, G. Smyrnis, M. Ivgi, M. Jordan, S. Gadre, H. Bansal, E. Guha, S. Keh, K. Arora, S. Garg, R. Xin, N. Muennighoff, R. Heckel, J. Mercat, M. Chen, S. Gururangan, M. Wortsman, A. Albalak, Y. Bitton, M. Nezhurina, A. Abbas, C.-Y. Hsieh, D. Ghosh, J. Gardner, M. Kilian, H. Zhang, R. Shao, S. Pratt, S. Sanyal, G. Ilharco, G. Daras, K. Marathe, A. Gokaslan, J. Zhang, K. Chandu, T. Nguyen, I. Vasiljevic, S. Kakade, S. Song, S. Sanghavi, F. Faghri, S. Oh, L. Zettlemoyer, K. Lo, A. El-Nouby, H. Pouransari, A. Toshev, S. Wang, D. Groeneveld, L. Soldaini, P. W. Koh, J. Jitsev, T. Kollar, A. G. Dimakis, Y. Carmon, A. Dave, L. Schmidt, and V. Shankar, "Datacomp-lm: In search of the next generation of training sets for language models," 2025. [Online]. Available: https://arxiv.org/abs/2406.11794

[4] E. Rahm and H. Do, "Data cleaning: Problems and current approaches," *IEEE Data Eng. Bull.*, vol. 23, pp. 3–13, 01 2000.

[5] I. F. Ilyas and X. Chu, *Data Cleaning*. New York, NY, USA: Association for Computing Machinery, 2019.

[6] W. Ni, X. Miao, X. Zhao, Y. Wu, S. Liang, and J. Yin, "Automatic data repair: Are we ready to deploy?" *Proc. VLDB Endow.*, vol. 17, no. 10, pp. 2617–2630, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p2617-miao.pdf

[7] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 458–469.

[8] Z. Khayyat, I. F. Ilyas, A. Jindal, S. Madden, M. Ouzzani, P. Papotti, J. Quiané-Ruiz, N. Tang, and S. Yin, "Bigdansing: A system for big data cleansing," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, T. K. Sellis, S. B. Davidson, and Z. G. Ives, Eds. ACM, 2015, pp. 1215–1230. [Online]. Available: https://doi.org/10.1145/2723372.2747646

[9] Y. Gao, C. Ge, X. Miao, H. Wang, B. Yao, and Q. Li, "A hybrid data cleaning framework using markov logic networks," 2019. [Online]. Available: https://arxiv.org/abs/1903.05826

[10] E. K. Rezig, M. Ouzzani, W. G. Aref, A. K. Elmagarmid, A. R. Mahmood, and M. Stonebraker, "Horizon: scalable dependency-driven data cleaning," *Proc. VLDB Endow.*, vol. 14, no. 11, p. 2546–2554, Jul. 2021. [Online]. Available: https://doi.org/10.14778/3476249.3476301

[11] M. Dallachiesa, A. Ebaid, A. Eldawy, A. Elmagarmid, I. F. Ilyas, M. Ouzzani, and N. Tang, "Nadeef: a commodity data cleaning system," in *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 541–552. [Online]. Available: https://doi.org/10.1145/2463676.2465327

[12] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "Holoclean: Holistic data repairs with probabilistic inference," *Proc. VLDB Endow.*, vol. 10, no. 11, pp. 1190–1201, 2017. [Online]. Available: http://www.vldb.org/pvldb/vol10/p1190-rekatsinas.pdf

[13] F. Chiang and R. J. Miller, "A unified model for data and constraint repair," in *Proceedings of the 27th International Conference on Data Engineering, ICDE 2011, April 11-16, 2011, Hannover, Germany*, S. Abiteboul, K. Böhm, C. Koch, and K. Tan, Eds. IEEE Computer Society, 2011, pp. 446–457. [Online]. Available: https://doi.org/10.1109/ICDE.2011.5767833

[14] G. Beskales, I. F. Ilyas, L. Golab, and A. Galiullin, "On the relative trust between inconsistent data and inaccurate constraints," 2012. [Online]. Available: https://arxiv.org/abs/1207.5226

[15] P. Bohannon, W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis, "Conditional functional dependencies for data cleaning," in *2007 IEEE 23rd International Conference on Data Engineering*, 2007, pp. 746–755.

[16] M. Yakout, L. Berti-Équille, and A. K. Elmagarmid, "Don't be scared: use scalable automatic repairing with maximal likelihood and bounded changes," in *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2013, New York, NY, USA, June 22-27, 2013*, K. A. Ross, D. Srivastava, and D. Papadias, Eds. ACM, 2013, pp. 553–564. [Online]. Available: https://doi.org/10.1145/2463676.2463706

[17] M. Mahdavi and Z. Abedjan, "Baran: Effective error correction via a unified context representation and transfer learning," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 1948–1961, 2020. [Online]. Available: http://www.vldb.org/pvldb/vol13/p1948-mahdavi.pdf

[18] M. Yan, Y. Wang, Y. Wang, X. Miao, and J. Li, "GIDCL: A graph-enhanced interpretable data cleaning framework with large language models," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 236:1–236:29, 2024. [Online]. Available: https://doi.org/10.1145/3698811

[19] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: A large language model for data preprocessing," 2024. [Online]. Available: https://arxiv.org/abs/2312.01678

[20] S. Mandt, M. D. Hoffman, and D. M. Blei, "Stochastic gradient descent as approximate bayesian inference," 2018. [Online]. Available: https://arxiv.org/abs/1704.04289

[21] A. Malinin and M. Gales, "Uncertainty estimation in autoregressive structured prediction," 2021. [Online]. Available: https://arxiv.org/abs/2002.07650

[22] J. S. Wang and P. M. Aronow, "Listwise deletion in high dimensions," *Political Analysis*, vol. 31, no. 1, p. 149–155, 2023.

[23] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, no. null, p. 1157–1182, Mar. 2003.

[24] L. Yu and H. Liu, "Efficient feature selection via analysis of relevance and redundancy," *J. Mach. Learn. Res.*, vol. 5, p. 1205–1224, Dec. 2004.

[25] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. V. Dillon, B. Lakshminarayanan, and J. Snoek, "Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift," 2019. [Online]. Available: https://arxiv.org/abs/1906.02530

[26] A. D'Amour, K. Heller, D. Moldovan, B. Adlam, B. Alipanahi, A. Beutel, C. Chen, J. Deaton, J. Eisenstein, M. D. Hoffman, F. Hormozdiari, N. Houlsby, S. Hou, G. Jerfel, A. Karthikesalingam, M. Lucic, Y. Ma, C. McLean, D. Mincu, A. Mitani, A. Montanari, Z. Nado, V. Natarajan, C. Nielson, T. F. Osborne, R. Raman, K. Ramasamy, R. Sayres, J. Schrouff, M. Seneviratne, S. Sequeira, H. Suresh, V. Veitch, M. Vladymyrov, X. Wang, K. Webster, S. Yadlowsky, T. Yun, X. Zhai, and D. Sculley, "Underspecification presents challenges for credibility in modern machine learning," 2020. [Online]. Available: https://arxiv.org/abs/2011.03395

[27] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[28] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[29] A. P. Dawid and A. M. Skene, "Maximum likelihood estimation of observer error-rates using the em algorithm," *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, vol. 28, no. 1, pp. 20–28, 1979. [Online]. Available: http://www.jstor.org/stable/2346806

[30] R. M. Neal and G. E. Hinton, *A View of the Em Algorithm that Justifies Incremental, Sparse, and other Variants*. Dordrecht: Springer Netherlands, 1998, pp. 355–368. [Online]. Available: https://doi.org/10.1007/978-94-011-5014-9_12

[31] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 39, no. 1, pp. 1–38, 1977. [Online]. Available: http://www.jstor.org/stable/2984875

[32] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves imagenet classification," 2020. [Online]. Available: https://arxiv.org/abs/1911.04252

[33] M. Ren, W. Zeng, B. Yang, and R. Urtasun, "Learning to reweight examples for robust deep learning," 2019. [Online]. Available: https://arxiv.org/abs/1803.09050

[34] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag, "Tabllm: Few-shot classification of tabular data with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2210.10723

[35] T. Dinh, Y. Zeng, R. Zhang, Z. Lin, M. Gira, S. Rajput, J. yong Sohn, D. Papailiopoulos, and K. Lee, "Lift: Language-interfaced fine-tuning for non-language machine learning tasks," 2022. [Online]. Available: https://arxiv.org/abs/2206.06565

[36] L. Jiang, Z. Zhou, T. Leung, L.-J. Li, and L. Fei-Fei, "Mentornet: Learning data-driven curriculum for very deep neural networks on corrupted labels," 2018. [Online]. Available: https://arxiv.org/abs/1712.05055

[37] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness, "Pseudo-labeling and confirmation bias in deep semi-supervised learning," 2020. [Online]. Available: https://arxiv.org/abs/1908.02983

[38] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS'13. Red Hook, NY, USA: Curran Associates Inc., 2013, p. 1196–1204.

[39] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Int. Res.*, vol. 70, p. 1373–1411, May 2021. [Online]. Available: https://doi.org/10.1613/jair.1.12125

[40] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," 2018. [Online]. Available: https://arxiv.org/abs/1610.02136

[41] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," 2017. [Online]. Available: https://arxiv.org/abs/1706.04599

[42] Q. C. Jianwei Wang, Wenjie Zhang, "Lasrepair++: Full version," https://github.com/OZCQC/LasRepair, 2026.

[43] P. C. Arocena, B. Glavic, G. Mecca, R. J. Miller, P. Papotti, and D. Santoro, "Messing up with bart: error generation for evaluating data-cleaning algorithms," *Proc. VLDB Endow.*, vol. 9, no. 2, p. 36–47, Oct. 2015. [Online]. Available: https://doi.org/10.14778/2850578.2850579

[44] D. B. Rubin, "Inference and missing data," *Biometrika*, vol. 63, no. 3, pp. 581–592, 1976. [Online]. Available: http://www.jstor.org/stable/2335739

[45] S. Krishnan, M. J. Franklin, K. Goldberg, and E. Wu, "Boostclean: Automated error detection and repair for machine learning," *CoRR*, vol. abs/1711.01299, 2017. [Online]. Available: http://arxiv.org/abs/1711.01299

[46] Y. Sasaki *et al.*, "The truth of the f-measure," *Teach tutor mater*, vol. 1, no. 5, pp. 1–5, 2007.

[47] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[48] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," 2023. [Online]. Available: https://arxiv.org/abs/1910.10683

[49] OpenAI, "Openai gpt-5 system card," 2026. [Online]. Available: https://arxiv.org/abs/2601.03267

[50] ——, "Gpt-4o system card," 2024. [Online]. Available: https://arxiv.org/abs/2410.21276

[51] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019. [Online]. Available: https://arxiv.org/abs/1910.13461

VIII. Appendix

A. Effectiveness of the EM algorithm

In Section III-B, we started Theorem 1, claiming that the EM refinement procedure is monotonic in the marginal log-likelihood.

We only need to consider the log-likelihood of the observed data at iteration t. Let $\{(\theta^{(t)}, \hat{\mathbf{z}}^{(t)})\}_{t\geq 0}$ be the sequence generated by the EM updates in Section III-B, where $\hat{\mathbf{z}}^{(t)} := \{\hat{z}_{ij}^{(t)}\}_{(i,j)\in E}$. The log-likelihood of observed data is:

$$\ell(\theta^{(t)}, z^{(t)}) = \log p_\theta(\mathbf{D}) = \sum_{(i,j)\in\mathcal{E}} \log p_\theta(z_{ij}^{(t)} \mid x_i^j).$$

Now we can restate the theorem and give the proof

Theorem 3 (Restatement of monotonicity of EM)**.** *Given the condition that both the Expectation and the maximization are ideal. We have the log-likelihood $\ell(\theta^{(t)}, z^{(t)})$ is monotonically non-decreasing along the iterations:*

$$\ell(\theta^{(t+1)}, \hat{\mathbf{z}}^{(t+1)}) \geq \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t)}), \qquad \forall t \geq 0.$$

Proof. We consider one Expectation step and one maximization step.

a) Expectation improves ℓ for fixed $\theta^{(t)}$.: At iteration t, given current parameter $\theta^{(t)}$, for each $(i,j) \in \mathcal{E}$. $\hat{z}_{ij}^{(t)} = \arg\max_z p_{\theta^{(t)}}(z \mid x_i^{(j)})$ maximizes the log-likelihood. Hence,

$$\begin{aligned}\ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t+1)}) &= \max_z p_{\theta^{(t)}}(z \mid x_i^{(j)}) \\ &\geq p_{\theta^{(t)}}(\hat{\mathbf{z}}^{(t)} \mid x_i^{(j)}) \\ &= \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t)}).\end{aligned}$$

b) maximization improves ℓ for fixed $\hat{\mathbf{z}}^{(t)}$.: By the definition of the maximization,

$$\theta^{(t+1)} = \arg\max_\theta \sum_{(i,j)\in\mathcal{E}} \log p_\theta(\hat{z}_{ij}^{(t)} \mid x_i^{(j)}).$$

Thus,

$$\begin{aligned}\ell(\theta^{(t+1)}, \hat{\mathbf{z}}^{(t+1)}) &= \max_\theta p_\theta(\hat{\mathbf{z}}^{(t+1)} \mid x_i^{(j)}) \\ &\geq p_{\theta^{(t)}}(\hat{\mathbf{z}}^{(t+1)} \mid x_i^{(j)}) \\ &= \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t+1)}).\end{aligned}$$

Combining the two inequalities yields

$$\ell(\theta^{(t+1)}, \hat{\mathbf{z}}^{(t+1)}) \;\geq\; \ell(\theta^{(t+1)}, \hat{\mathbf{z}}^{(t)}) \;\geq\; \ell(\theta^{(t)}, \hat{\mathbf{z}}^{(t)}),$$

which proves the monotonicity. □

Notice that we are using a weighted maximization in LasRepair++. If the maximization uses row-level weights $w_i^{(t)}$ (computed after the Expectation) and maximizes $\sum_{(i,j)\in\mathcal{E}} w_i^{(t)} \log p_\theta(\hat{z}_{ij}^{(t)} \mid x_i^{(j)})$, the same block-coordinate argument shows monotonic non-decrease of the corresponding weighted objective within iteration t (with $w^{(t)}$ treated as fixed during the maximization), so we omit the similar proof here.

B. Guarantees of Confident Learning

We now analyze the effectiveness of the SLM. We analyze one fixed EM iteration and omit the superscript t for simplicity. Recall that for each detected erroneous cell $(i,j) \in \mathcal{E}$, we extract the top-K first-token candidates and their probabilities $q_{ij}(r)$, compute a token-dependent threshold $T_j(v)$, and keep a candidate only if its probability exceeds the corresponding threshold. The corresponding probability is then used as the cell confidence c_{ij}, and row weights w_i are obtained by aggregating $\{c_{ij}\}_{j=1}^d$. Then, maximization is performed to maximize the row-weighted log-likelihood.

Theorem 4 (Threshold reduces error rate)**.** *Assume (Ranking consistency) that within each predicted first token, the softmax probability q is an order-consistent score: a higher q should not correspond to a higher error probability. This is a weak condition and can be empirically verified.*

The threshold $T_j(v)$ reduces the conditional error probability, that is:

$$P(\hat{z}_{ij} \neq z_{ij} \mid q_{ij} \geq T_j(v), \hat{a}_{ij} = v) \leq P(\hat{z}_{ij} \neq z_{ij} \mid \hat{a}_{ij} = v).$$

We treat the above probability as a function of q for the following proof.

Proof. Let A be the event "$\hat{z}_{ij} \neq z_{ij}$", B, C be the event "$q_{ij} \geq T_j(v)$" and "$\hat{a}_{ij} = v$". Let $f_q(\cdot)$ be the probability mass function of q_{ij}. By the law of total probability, we have

$$P(A \mid C) = P(B)P(A \mid B, C) + P(B^c)P(A \mid B^c, C).$$

For simplicity, let $n = P(q_{ij} \geq T_j(v)) \in (0,1)$ and define

$$m_1 = P(A \mid B, C), \qquad m_2 = P(A \mid B^c, C).$$

Then,

$$P(A \mid C) = nm_1 + (1-n)m_2,$$

and we have

$$P(A \mid C) - P(A \mid B, C) = (1-n)(m_2 - m_1).$$

Using the ranking consistency, $P(\hat{z}_{ij} \neq z_{ij} \mid q_{ij} = q, \hat{a}_{ij} = v)$ is non-increasing in q. We denote it by $g(q)$, then m_1 and m_2 are the conditional expectation:

$$\begin{aligned}m_1 &= \frac{\int_{T_j(v)}^1 P(A \mid q_{ij} = q, C) f_q(q)\mathrm{d}q}{\int_{T_j(v)}^1 f_q(q)\mathrm{d}q} = \mathbb{E}(g(q) \mid q \geq T_j(v), C) \\ &\leq g(T) \leq \mathbb{E}(g(q) \mid q \leq T_j(v), C) = m_2.\end{aligned}$$

Thus $m_2 - m_1 \geq 0$, and we have proved $P(A \mid C) \geq P(A \mid B, C)$, which is

$$P(\hat{z}_{ij} \neq z_{ij} \mid q_{ij} \geq T_j(v), \hat{a}_{ij} = v) \leq P(\hat{z}_{ij} \neq z_{ij} \mid \hat{a}_{ij} = v),$$

Exactly the desired inequality. □

We also demonstrate that the weighted data yields a gradient closer to the true gradient for parameter updates compared to the original data.

Theorem 5 (Restatement of weighted update effectiveness)**.** *For each row i, define the log-likelihood loss*

$$\hat{\ell}_i(\theta) = \sum_{(i,j)\in\mathcal{E}} -\log p_\theta(\hat{z}_{ij} \mid x_i^{(j)}),$$

and the corresponding (unknown clean label) loss

$$\ell_i^*(\theta) = \sum_{(i,j)\in\mathcal{E}} -\log p_\theta(z_{ij} \mid x_i^{(j)}).$$

Let $\hat{g}_i(\theta) = \nabla_\theta \hat{\ell}_i(\theta)$ *and* $g_i^*(\theta) = \nabla_\theta \ell_i^\star(\theta)$. *Assume there is a heteroscedastic noise decomposition*

$$\hat{g}_i(\theta) = g_i^*(\theta) + \epsilon_i,$$

where $\epsilon_i \sim N(\mu_i, \sigma^2/\omega_i)$, *and* $\{\epsilon_i\}_{i=1}^n$ *are conditionally independent. Without loss of generality, we can assume* $\mu_i = 0$. *Recall that* ω_i^* *is the normalized row-level weight. Define the weighted gradient* $\hat{g}_\omega$, *unweighted (uniform weighted) gradient* $\hat{g}_u$ *and the true gradient* g^* *as*

$$\hat{g}_\omega(\theta) = \sum_{i=1}^n \omega_i^* \hat{g}_i(\theta), \quad \hat{g}_u(\theta) = \frac{1}{n}\sum_{i=1}^n \hat{g}_i(\theta), \quad g^*(\theta) = \frac{1}{n}\sum_{i=1}^n g_i^*(\theta).$$

Then, for any fixed θ,

$$\mathbb{E}\|\hat{g}_\omega(\theta) - g^*(\theta)\| \le \mathbb{E}\|\hat{g}_u(\theta) - g^*(\theta)\|.$$

Proof. With θ fixed, we have

$$\mathbb{E}\|\hat{g}_u(\theta) - g^*(\theta)\| = \frac{1}{n^2}\sum_{i=1}^n \mathbb{E}\|\epsilon_i\| = \frac{\sigma^2}{n^2}\sum_{i=1}^n \frac{1}{\omega_i^*}.$$

Similarly,

$$\mathbb{E}\|\hat{g}_\omega(\theta) - g^*(\theta)\| = \sum_{i=1}^n \mathbb{E}(\omega_i^{*2}\epsilon_i^2) = \sigma^2.$$

By the Cauchy-Schwarz inequality,

$$\left(\sum_{i=1}^n \omega_i^*\right)\left(\sum_{i=1}^n \frac{1}{\omega_i^*}\right) \ge n^2 \quad \Longrightarrow \quad 1 \le \frac{1}{n^2}\sum_{i=1}^n \frac{1}{\omega_i^*}.$$

Finally we proved

$$\mathbb{E}\|\hat{g}_\omega(\theta) - g^*(\theta)\| \le \mathbb{E}\|\hat{g}_u(\theta) - g^*(\theta)\|.$$

The equality case happens when $\omega_1 = \cdots = \omega_n$. □